

SampleManager LIMS Ingestion Pipeline — Flow Documentation

Metadata-driven pipeline that lands daily LIMS extracts from a legacy Oracle (Thermo SampleManager), transforms them to Parquet, exposes them to Redshift via Spectrum, loads a typed data warehouse, and serves reporting views to SAC.

The pipeline is **config-driven**: adding or changing a source table is a config change, not a code change. The same code path serves every table and both factories (**DKQ** and **FOS**).

1. End-to-end flow at a glance

Legacy Oracle (SampleManager)

| daily full extracts (SQL*Plus spool, Windows-1252, .csv.gz)



S3 RAW / landed zone	raw/{DKQ FOS}/TABLE_prod_extract.csv.gz
----------------------	---



▼ Step Function: prsm-sfn-dev-samplemanager-run

1. Config Router (Lambda)	scan landed zone, match each file to its config, emit per-file parameters
---------------------------	--

| Files[] (fan-out, MaxConcurrency 13)



2. Map → Parallel	per file, two branches in PARALLEL
A. Copy to replicated	→ S3 replicated mirror (raw backup)
B. Glue transform job	→ S3 PROCESSED zone (Snappy Parquet, partitioned) + Glue Data Catalog table (raw DB)

| (all files done)



3. Redshift Loader (Lambda)	Redshift Spectrum ext schema reads the Parquet DELETE + INSERT...SELECT (cast to typed columns) into the DW schema – one atomic transaction
-----------------------------	---



4. Choice: load OK?	200 → Succeed else → Fail
---------------------	-----------------------------



Reporting views (DW) → SAP Analytics Cloud (SAC)

2. Landing zone & naming conventions

- Extracts land under the **raw/landed** S3 base, in a **per-factory subfolder**: `... /DKQ/...` and `... /FOS/...`. The subfolder name **is** the source-system tag — it is not stored in the file or the config.
- Filenames carry an environment suffix, e.g. `FOURNISSEUR_prod_extract.csv.gz`. The router strips the compound extension (`.csv.gz`), the `_prod_extract` / `_qa_extract` / `_dev_extract` suffix, and

any `DKQ_` / `FOS_` prefix to recover the **bare table name** (`FOURNISSEUR`) used to find the matching config.

- Loads are **full drop-reload each day** (not incremental). `source` (the factory) is the only partition key.

3. Configs — the metadata that drives everything

One JSON per source table in `configs/`, keyed by the bare table name (`SAMPLE.json`, `PHRASE.json`, ...). Minimal by design:

```
{ "TargetTable": "samp_test_result_lafa", "HasHeader": "true",
  "Delimiter": ",", "Encoding": "windows-1252" }
```

Field	Meaning
TargetTable	Catalog / DW table name the file loads into
HasHeader	Whether the CSV has a header row
Delimiter	Field delimiter
Encoding	Source encoding (legacy Oracle emits windows-1252 ; default utf-8)

A file with **no matching config is skipped** (logged, not fatal). There are currently **15 configs** — that set defines what the pipeline ingests.

4. Step Function — orchestration

`AWS/prsm-sfn-dev-samplemanager-run.asl`

#	State	Type	What it does
1	DiscoverAndConfigLambda	Task → Lambda	Runs the config router; outputs the <code>Files[]</code> work list.
2	ProcessFilesMap	Map (concurrency 13)	One iteration per file; each iteration is a Parallel block.
2A	CopyToReplicatedVault	Task → S3 CopyObject	Copies the landed file into the replicated mirror (raw backup / lineage).
2B	ExecuteGlueTransformation	Task → Glue <code>.sync</code>	Runs the Glue job and waits for it to finish.
3	LoadRedshiftDw	Task → Lambda	Invokes the loader with <code>{ "table_name": "all" }</code> .
4	CheckDwLoadResult	Choice	Loader <code>statusCode == 200</code> → Succeed , otherwise → Fail .

The two branches in step 2 are independent: backup and transform run at the same time for each file. The Map only advances to the loader once **all** files have finished both branches.

5. Component detail (what each piece does)

5.1 Config Router Lambda — `lims-config-router-lambda`

Entry point / dispatcher. - Paginates the landed zone (handles >1000 objects). - For each file: derives the bare table name, reads its config, and builds: - **Replicated copy params** — mirror the landed layout `replicated/{DKQ|FOS}/{file}` (same key each run → overwrites yesterday's copy). - **Glue job arguments** — source S3 path, processed S3 path (root table folder), target table, source system (=subfolder), raw catalog DB name, and the header/delimiter/encoding from the config. - Returns `{ "Files": [ { ReplicatedCopyParams, GlueArguments }, ... ] }`. - All buckets/paths come from **environment variables** (`RAW_S3_BASE`, `REPLICATED_S3_BASE`, `CONFIG_S3_BASE`, `PROCESSED_S3_BASE`, `ENVIRONMENT`) so promotion dev→qa→prod needs no code change.

5.2 Glue Job — `lims_ingestion_job` (PySpark)

Raw CSV → catalogued Parquet. Runs once per file. - Reads the CSV **as all-string** (no schema inference) using the configured delimiter/encoding/header; quotes and escapes handled explicitly. - Adds two columns: `source` (the factory) and `last_updated_at` (run timestamp). - Writes **Snappy Parquet partitioned by source** into the processed zone and registers the table in the Glue Data Catalog (raw DB `prsm_glb_{env}_samplemanager_raw`). - Steady-state runs are engineered for **zero Glue table-version churn** (the account-wide 1M `TABLE_VERSION` limit was hit before): on re-runs it deletes only *this* source's partition objects from S3 and rewrites Parquet in place — no `MSCK` / `ALTER TABLE` when the partition already exists. - Safety handling built in: **schema-drift** recreate (catalog columns no longer match the incoming file), **new-partition** registration (first time a factory appears), **concurrent DKQ/FOS race** on first create, old table-version cleanup, and removal of EMRFS `_${folder$}` markers.

5.3 Redshift Loader Lambda — `lims-redshift-loader-lambda`

Spectrum (ext) → typed DW. Invoked with `table_name: "all"` (or a single table). - The **ext / Spectrum schema** (`prsm_edhid_samplemanager_ext`) exposes the Glue Parquet tables with every column as string. - For each table: **DELETE** the DW table, then **INSERT ... SELECT** from the ext table, **casting each column** to its typed DW type (numeric via `NULLIF(TRIM( ... )) → BIGINT`). The `source` partition fills `SOURCE`; the Glue timestamp fills `LAST_UPDATED_AT`. - **One transaction, single commit** = atomic. **DELETE** (not **TRUNCATE**) is deliberate: if any table fails, nothing commits and **yesterday's data survives**. - Tables whose ext table doesn't exist yet (source not delivered) are **skipped and reported**, not failed. - **Row filter (temporary)**: the two fact extracts can arrive corrupt (Oracle spool error text lands as junk rows); rows whose `id_numeric` isn't a clean integer are dropped so the load still succeeds. Remove when the source is fixed. - Credentials: env vars (dev override) → else Secrets Manager (`SecretId`).

6. Data model & schemas

Layer	Location	Populated by
Landed (raw CSV)	S3 <code>RAW_S3_BASE/{DKQ,FOS}/...</code>	Upstream Oracle extract
Replicated mirror	S3 <code>REPLICATED_S3_BASE/{DKQ,FOS}/...</code>	Step 2A (S3 copy)
Processed (Parquet)	S3 <code>PROCESSED_S3_BASE/{table}/source=...</code>	Glue job
Glue Data Catalog	DB <code>prsm_glb_{env}_samplemanager_raw</code>	Glue job
Redshift ext (Spectrum)	schema <code>prsm_edhid_samplemanager_ext</code>	Reads Glue Parquet
Redshift DW (typed)	schema <code>prsm_edhid_samplemanager_dw</code>	Loader Lambda

15 modeled tables (DDL in `DDL/`, typed DW DDL in `DDL/latest_dll_truth/ALL_TABLES_REDSHIFT.sql`):

`versioned_analysis`, `versioned_component`, `mlp_header`, `sample`, `sample_point`, `customer`, `location`, `code_controle`, `fournisseur`, `job_header`, `phrase`, `mensuel`, `samp_test_result_lafa`, `c_samp_test_result_lafa`, `c_sample`.

- `samp_test_result_lafa` and `c_samp_test_result_lafa` are the flat fact tables (identical 81-column structure). The plain table holds the live rolling window; the `c_` (completed/archived) table holds older authorised samples. The two row-sets are disjoint, so reporting unions both to get full history.

7. Reporting layer (DW views → SAC)

Built on top of the DW schema, consumed by SAP Analytics Cloud:

- `vw_lafa_report_base` — the single base view. Unions the two fact tables and left-joins the dimensions it needs. It reads values from **7 tables**: `samp_test_result_lafa`, `c_samp_test_result_lafa`, `code_controle`, `mlp_header`, `job_header`, `versioned_analysis`, `phrase`.
- `vw_bilan_kk_dkq` and `vw_recherche_simplifiee` — per-report wrappers that select *only* from `vw_lafa_report_base` (no direct base-table access).

Not used by reporting: `customer`, `c_sample`, `fournisseur`, `location`, `mensuel`, `sample`, `sample_point`, `versioned_component` are ingested but never read by any view — the fact tables are denormalized and already carry those codes as columns. (`mlp_view` / `phrase_format` are referenced only in commented-out joins pending future ingestion.)

8. Operational notes

- **Idempotent / rerunnable:** full reload every run; a failed DW load leaves the previous day intact.
- **Adding a source table:** drop the extract in the landed folder and add a `configs/TABLE.json`; add the typed table to the loader's `TABLES` map + DDL if it needs a DW table. No orchestration change.

- **Both factories share everything:** same config, same code; `source` (DKQ/FOS) is the only differentiator and the partition key.
- **Environments:** all paths/DB names are env-var driven, so dev/qa/prod differ only in configuration. ``